Índice

1. Descripción del Taller **2**

2. Desarrollo de los Ejercicios **2**
2.1. 1. Inversión de Colores 2
2.2. 2. Blur (Desenfoque) 3
2.3. 3. Recorte de Imagen 3
2.4. 4. Imagen Binaria 4
2.5. 5. Detección de Bordes (Sobel) 5

3. Análisis de Rendimiento **5**
3.1. Ejercicio 1 5
3.2. Ejercicio 2 6
3.3. Ejercicio 3 6
3.4. Ejercicio 4 6
3.5. Ejercicio 5 6

4. Desarrollo del Proyecto con Paralelización **7**

5. Procesamiento de Video **7**

6. Resultados **7**

7. Informe de Contribución Individual **8**
7.1. Dylan 8
7.2. Jonathan 8
7.3. Steeven 9

8. Tabla de Evaluación Interna **9**

9. Repositorio de Ejecución en Google Colab **9**

10. Conclusiones Generales **10**

1. Descripción del Taller

Este laboratorio tiene como objetivo implementar cinco filtros de procesamiento de imágenes utilizando CUDA y aplicarlos al procesamiento de video.

A diferencia del taller anterior, donde los filtros se aplicaban sobre imágenes individuales, en este laboratorio los algoritmos desarrollados se extienden al procesamiento de video, considerando el video como una secuencia de imágenes o frames.

Cada frame del video es procesado de manera independiente en la GPU mediante kernels CUDA. Posteriormente, los frames procesados son ensamblados nuevamente para generar el video resultante.

Los filtros desarrollados en este taller fueron los siguientes:

- Inversión de colores (Negativo)
- Blur (Desenfoque)
- Recorte de imagen
- Imagen binaria (Threshold)
- Detección de bordes (Sobel)

Criterios de evaluación:

- Correcta ejecución de los algoritmos
- Código limpio y organizado
- Informe técnico del laboratorio

2. Desarrollo de los Ejercicios

2.1. 1. Inversión de Colores

Fórmula implementada:

$$R' = 255 - R$$
$$G' = 255 - G$$
$$B' = 255 - B$$

Descripción de implementación:

El ejercicio 1 consistió en la implementación del filtro de inversión de colores (negativo) utilizando programación paralela en CUDA. Este filtro es una operación puntual, ya que el nuevo valor de cada píxel depende únicamente de su valor original y no requiere información de píxeles vecinos.

Dentro de la implementación para video, este filtro fue aplicado a cada frame de manera independiente. Se diseñó un kernel donde cada hilo procesa un único píxel del frame. La posición del píxel fue calculada mediante los índices globales de hilo en la malla de ejecución.

Una vez validado que las coordenadas se encontraban dentro de los límites del frame, se aplicó la operación de inversión para cada canal RGB. La estructura general del programa incluyó la lectura secuencial del video, la extracción de frames, la reserva de memoria en GPU, la copia de cada frame desde el host al device, la ejecución del kernel y la copia del resultado de vuelta al host para la reconstrucción del video final.

2.2. 2. Blur (Desenfoque)

Kernel utilizado:

$$P_{out}(i,j) = \frac{1}{(2r+1)^2} \sum_{m=-r}^{r} \sum_{n=-r}^{r} P_{in}(i+m, j+n)$$

Descripción de implementación:

El ejercicio 2 consistió en la implementación del filtro de desenfoque (Blur) mediante una operación de convolución discreta utilizando programación paralela en CUDA. A diferencia del filtro negativo, este algoritmo requiere información de los píxeles vecinos para calcular el nuevo valor de cada píxel.

Se diseñó un kernel en el que cada hilo procesa un píxel del frame de salida. Para cada posición (i, j), el hilo recorre una vecindad cuadrada de tamaño $(2r+1) \times (2r+1)$ alrededor del píxel central y acumula los valores de intensidad correspondientes a los canales R, G y B. Posteriormente, la suma total se divide entre el número de elementos del kernel $(2r+1)^2$ con el fin de obtener el promedio.

Para evitar accesos inválidos a memoria cuando el píxel se encuentra cerca de los bordes del frame, se implementó una técnica de manejo de límites (*clamping*), ajustando las coordenadas fuera de rango al píxel válido más cercano. Finalmente, se realizó la transferencia de datos desde el host hacia el device, la ejecución del kernel y la copia del resultado final nuevamente al host para su incorporación en el video procesado.

2.3. 3. Recorte de Imagen

Relación de mapeo:

$$P_{dest}(i,j) = P_{src}(i + x_{start}, j + y_{start})$$

Descripción de implementación:
El ejercicio 3 consistió en implementar el recorte de cada frame del video a partir de una región de interés definida por el usuario. El recorte fue tratado como una operación de copia selectiva de datos desde la imagen original hacia una imagen de salida de menor tamaño.

Cada hilo de la GPU fue encargado de copiar un píxel de la región recortada. Para ello, se utilizó el desplazamiento definido por (x_{start}, y_{start}), garantizando que la posición de lectura en la imagen fuente correspondiera con la posición de escritura en la imagen destino.

El manejo de límites fue una parte crítica del desarrollo, especialmente en los filtros de convolución (Blur y Sobel) y en el recorte de imagen.

En GPU, cada hilo accede directamente a posiciones de memoria basadas en su índice calculado. Si no se validan correctamente los índices, el kernel puede intentar acceder a posiciones fuera del arreglo, provocando:

- Lectura de memoria no asignada.
- Resultados incorrectos.
- Error de ejecución en la GPU.

Para evitar esto, se implementó una validación condicional al inicio de cada kernel:

$$\texttt{if}(x < width \texttt{ and } y < height)$$

De esta manera, solo los hilos cuyos índices estén dentro de las dimensiones válidas del frame ejecutan el procesamiento.

En filtros de convolución (Blur y Sobel) Como estos filtros requieren acceder a una vecindad 3×3 alrededor del píxel central, fue necesario asegurar que el píxel no estuviera en el borde del frame.

Se implementó la validación:

$$\texttt{if}(x > 0 \texttt{ and } x < width - 1 \texttt{ and } y > 0 \texttt{ and } y < height - 1)$$

Esto garantiza que todos los vecinos necesarios existen dentro de la imagen.

En los píxeles de borde se tomó la decisión de:

- No aplicar convolución y conservar el valor original, o
- Asignar un valor neutro, según el filtro.

En el recorte de imagen En el recorte se validó que la región de interés (ROI) no excediera las dimensiones originales:

$$x_{start} + width_{crop} \leq width_{original}$$
$$y_{start} + height_{crop} \leq height_{original}$$

Además, dentro del kernel se validó:

$$\texttt{if}(i + x_{start} < width_{original} \texttt{ and } j + y_{start} < height_{original})$$

Esto evitó accesos inválidos cuando el usuario definía parámetros de recorte cercanos a los bordes.

Impacto en rendimiento Aunque las validaciones introducen instrucciones condicionales adicionales, su costo es mínimo comparado con el beneficio de evitar fallos de memoria y garantizar estabilidad del kernel.

El correcto manejo de límites permitió que todos los filtros funcionaran de manera robusta y segura para frames de distintos tamaños.

2.4. 4. Imagen Binaria

Regla lógica implementada:

$$P_{out}(i,j) = \begin{cases} 255 & \text{si } \dfrac{R(i,j) + G(i,j) + B(i,j)}{3} > T \\ 0 & \text{si } \dfrac{R(i,j) + G(i,j) + B(i,j)}{3} \leq T \end{cases}$$

Descripción de implementación:

El ejercicio 4 consistió en la implementación del proceso de binarización utilizando programación paralela en CUDA. Este algoritmo es una operación puntual, ya que el valor de salida de cada píxel depende únicamente de su intensidad y del valor del umbral T.

Se diseñó un kernel en el que cada hilo de la GPU procesa un único píxel del frame. Primero, el hilo calcula la posición global correspondiente mediante los índices del bloque y del hilo. Posteriormente, se leen los valores R, G y B del píxel y se calcula la intensidad promedio.

Una vez obtenida la intensidad, se compara con el umbral definido por el usuario. Si la intensidad es mayor que T, se asigna el valor 255 (blanco); en caso contrario, se asigna 0 (negro).

Dentro del flujo de video, este procedimiento se repitió sobre cada frame del archivo de entrada, preservando la secuencia temporal para la posterior reconstrucción del video final. El umbral utilizado para las pruebas fue $T = 128$, correspondiente al punto medio del rango de intensidades.

2.5. 5. Detección de Bordes (Sobel)

Cálculo del gradiente implementado:

$$G(x, y) = \sqrt{G_x(x, y)^2 + G_y(x, y)^2}$$

Descripción de implementación:

El ejercicio 5 consistió en la implementación del operador Sobel para la detección de bordes mediante programación paralela en CUDA. Este filtro se basa en la estimación del gradiente de intensidad en dos direcciones principales.

Como paso inicial, cada frame RGB fue convertido a escala de grises utilizando el promedio de los canales:

$$I(x, y) = \frac{R(x, y) + G(x, y) + B(x, y)}{3}$$

Posteriormente, se aplicaron dos máscaras de convolución 3×3 correspondientes a las derivadas aproximadas en dirección horizontal (G_x) y vertical (G_y).

Cada hilo de la GPU procesó un píxel del frame de salida, calculando la suma ponderada de su vecindad inmediata según los coeficientes de los kernels Sobel. Una vez obtenidos G_x y G_y, se calculó la magnitud del gradiente utilizando la norma Euclidiana.

El valor resultante fue limitado al rango $[0, 255]$ para garantizar que pudiera representarse como imagen. Además, se implementó validación de límites para evitar accesos fuera de memoria en los píxeles ubicados en los bordes del frame.

La estructura general del programa incluyó la lectura del video, el procesamiento frame a frame, la reserva de memoria en GPU, la copia de datos desde el host al device, la ejecución del kernel y la recuperación del resultado final para la reconstrucción del video.

3. Análisis de Rendimiento

3.1. Ejercicio 1

El filtro de inversión de colores presentó un rendimiento altamente eficiente en GPU debido a su naturaleza completamente paralelizable. Al ser una operación puntual sin dependencias entre píxeles, cada hilo ejecuta una operación aritmética simple e independiente, lo que permite aprovechar plenamente el paralelismo masivo de CUDA.

En comparación con una implementación secuencial en CPU, donde se utilizan bucles anidados para recorrer todos los píxeles de cada frame, la versión en GPU reduce significativamente el tiempo de ejecución para videos de duración moderada y resolución media o alta. El costo computacional por hilo es mínimo, por lo que el principal factor de tiempo corresponde a la transferencia de memoria entre host y device.

3.2. Ejercicio 2

El filtro de desenfoque presentó una mejora significativa en el rendimiento al ser implementado en GPU mediante CUDA. Debido a que este algoritmo se basa en una operación de convolución, cada píxel requiere múltiples operaciones aritméticas asociadas al promedio de sus vecinos dentro del kernel definido.

En una implementación secuencial en CPU, este proceso se realiza mediante bucles anidados sobre cada frame y sobre la vecindad de cada píxel, lo cual incrementa considerablemente el tiempo de ejecución. Sin embargo, en la implementación paralela, cada hilo de la GPU se encarga del procesamiento de un píxel de forma independiente, permitiendo ejecutar múltiples cálculos simultáneamente.

Aunque el algoritmo requiere validaciones adicionales para el manejo de límites en los bordes del frame, el impacto de estas instrucciones condicionales es mínimo frente al beneficio obtenido por la paralelización masiva.

3.3. Ejercicio 3

En el ejercicio de recorte de imagen, el rendimiento mejoró significativamente al utilizar paralelización con CUDA, ya que cada hilo fue encargado de copiar un único píxel desde el frame original hacia la imagen destino utilizando el desplazamiento definido por (x_{start}, y_{start}).

En la versión secuencial en CPU, el recorte se realiza mediante bucles anidados que recorren toda la región de interés de cada frame, mientras que en GPU el trabajo se distribuye entre múltiples hilos ejecutándose simultáneamente. Debido a que el recorte es una operación de copia directa sin cálculos complejos ni convoluciones, el costo computacional por hilo es mínimo, lo que permite una ejecución altamente eficiente.

3.4. Ejercicio 4

El filtro de binarización presentó un rendimiento altamente eficiente en GPU debido a su naturaleza completamente paralelizable. Al tratarse de una operación puntual, cada hilo ejecuta un número reducido de operaciones aritméticas independientes, como el cálculo del promedio y la comparación con el umbral.

En la versión secuencial en CPU, el algoritmo requiere recorrer todos los píxeles de cada frame mediante bucles anidados, mientras que en la implementación en CUDA el procesamiento se distribuye entre múltiples hilos que operan simultáneamente sobre distintos píxeles.

El costo computacional por hilo es mínimo, por lo que el tiempo total de ejecución depende principalmente del tamaño de cada frame, de la duración del video y del costo de transferencia de memoria entre host y device.

3.5. Ejercicio 5

El filtro de detección de bordes mediante Sobel presentó un mayor costo computacional en comparación con los ejercicios anteriores, debido a que requiere múltiples operaciones de convolución por píxel.

Cada hilo debe acceder a una vecindad 3×3 y realizar varias multiplicaciones y sumas para calcular G_x y G_y, seguido del cálculo de la raíz cuadrada para obtener la magnitud del gradiente. En una implementación secuencial en CPU, esto implica una cantidad significativa de operaciones repetitivas sobre todos los frames del video.

Sin embargo, gracias a la paralelización en CUDA, cada píxel es procesado simultáneamente por distintos hilos, lo que reduce considerablemente el tiempo de ejecución en videos con mayor resolución o mayor número de frames. Aunque el manejo de límites introduce condiciones adicionales, su impacto en rendimiento es bajo frente al beneficio del paralelismo masivo.

4. Desarrollo del Proyecto con Paralelización

La paralelización del proyecto se implementó mediante el uso de CUDA, permitiendo distribuir el procesamiento de los píxeles de cada frame entre múltiples hilos ejecutados simultáneamente en la GPU. En cada uno de los filtros desarrollados, se diseñó un kernel donde cada hilo calcula el valor correspondiente a un píxel de la imagen de salida.

Para ello, se realizó la configuración de la malla de ejecución (*grid* y *blocks*), el cálculo de índices globales de cada hilo y la validación de límites para evitar accesos fuera de memoria. Adicionalmente, se gestionó la transferencia de datos entre la memoria del host (CPU) y del device (GPU), así como la reserva de memoria necesaria para almacenar los resultados.

Este enfoque permitió reducir el tiempo de ejecución de los algoritmos y aprovechar el paralelismo inherente al procesamiento de imágenes digitales, donde cada píxel puede ser tratado de forma independiente o con dependencia local. En el contexto del presente taller, esta misma lógica fue extendida al procesamiento de video mediante el tratamiento independiente de cada frame.

5. Procesamiento de Video

Para la implementación del presente taller se utilizó un enfoque basado en el procesamiento de video mediante extracción de frames. Un video digital puede interpretarse como una secuencia ordenada de imágenes, donde cada imagen corresponde a un frame del video.

El flujo general del procesamiento se implementó de la siguiente manera:

1. Carga del video original mediante librerías de procesamiento de video.
2. Extracción secuencial de cada frame del video.
3. Transferencia de cada frame a la memoria de la GPU.
4. Aplicación del filtro CUDA correspondiente sobre cada frame.
5. Recuperación del frame procesado desde la GPU hacia la CPU.
6. Almacenamiento del frame procesado.
7. Reconstrucción del video final utilizando la secuencia de frames resultantes.

Este enfoque permitió reutilizar directamente los kernels desarrollados para procesamiento de imágenes individuales, ya que cada frame es tratado como una imagen independiente. De esta manera, se mantiene el modelo de paralelización por píxel implementado en el taller anterior.

La ejecución en GPU permite que cada píxel del frame sea procesado simultáneamente por diferentes hilos de ejecución, aprovechando el paralelismo masivo de CUDA y mejorando significativamente la eficiencia del procesamiento.

6. Resultados

Durante la ejecución del proyecto se aplicaron los diferentes filtros sobre un video de prueba utilizando el entorno Google Colab con aceleración por GPU. Los resultados obtenidos evidencian el comportamiento esperado de cada uno de los filtros:

- El filtro de inversión de colores genera el negativo visual del video.

- El filtro Blur suaviza las transiciones de intensidad reduciendo ruido de alta frecuencia.
- El recorte de imagen permite seleccionar regiones específicas del video original.
- La binarización transforma el video en una representación blanco y negro basada en un umbral.
- El operador Sobel permite identificar los bordes principales de los objetos presentes en el video.

La implementación demostró que los algoritmos desarrollados en CUDA pueden aplicarse eficientemente sobre grandes volúmenes de datos visuales, como lo son los videos, manteniendo tiempos de ejecución razonables gracias al paralelismo de la GPU.

7. Informe de Contribución Individual

7.1. Dylan

Funciones implementadas:

Dylan se encargó del desarrollo del segundo ejercicio correspondiente al filtro de desenfoque (Blur). Esto incluyó la implementación del kernel de convolución en CUDA, la configuración de la malla de hilos (*grid* y *blocks*), la gestión de memoria en GPU mediante copias host-device y la integración del procesamiento del filtro sobre los frames del video.

Dificultades y soluciones:

Durante el desarrollo del filtro Blur se presentaron dificultades relacionadas con el manejo de los píxeles ubicados en los bordes del frame. Al aplicar la convolución, algunos vecinos requeridos para el cálculo del promedio podían encontrarse fuera de los límites válidos del arreglo, generando accesos inválidos a memoria en la GPU.

Para solucionar este inconveniente, se implementó una técnica de *clamping*, ajustando las coordenadas fuera de rango al píxel válido más cercano dentro del frame. Esto permitió garantizar la estabilidad del kernel y evitar errores de ejecución, manteniendo la integridad del resultado final.

7.2. Jonathan

Funciones implementadas:

Jonathan se encargó del desarrollo del cuarto y quinto ejercicio correspondientes al filtro de Imagen Binaria (Threshold) y a la Detección de Bordes mediante el operador Sobel.

En el ejercicio de Imagen Binaria, se implementó un kernel en CUDA donde cada hilo procesa un píxel individual, calcula su intensidad promedio a partir de los canales RGB y la compara con un umbral definido. Dependiendo del resultado de la comparación, se asigna el valor 0 o 255 en la imagen de salida.

En el ejercicio de Sobel, se desarrolló la conversión previa a escala de grises y posteriormente la implementación del operador Sobel utilizando dos máscaras de convolución 3×3 para calcular las componentes del gradiente G_x y G_y. Finalmente, se calculó la magnitud del gradiente mediante la expresión:

$$G = \sqrt{G_x^2 + G_y^2}$$

También se configuró la malla de ejecución (*grid* y *blocks*), la gestión de memoria en GPU y la transferencia de datos entre host y device para el procesamiento frame a frame del video.

Dificultades y soluciones:
En el ejercicio de Imagen Binaria, el principal reto fue garantizar que el procesamiento se realizara correctamente sobre frames en formato RGB y que la comparación con el umbral se mantuviera dentro del rango válido de intensidades.

En el ejercicio de Sobel, la mayor dificultad estuvo relacionada con el manejo de los píxeles ubicados en los bordes del frame, ya que el operador requiere acceder a una vecindad 3×3. Para evitar accesos inválidos a memoria en la GPU, se implementaron validaciones condicionales que aseguran que únicamente se procesen píxeles con vecinos válidos dentro de los límites de la imagen.

7.3. Steeven

Funciones implementadas:
Steeven se encargó del desarrollo completo del primer ejercicio (Inversión de Colores) y del tercer ejercicio (Recorte de Imagen). Esto incluyó la implementación del kernel en CUDA, la configuración de la malla de hilos (*grid* y *blocks*), la gestión de memoria en GPU y la validación de los resultados obtenidos sobre cada frame del video.

Dificultades y soluciones:
En el tercer ejercicio se presentaron dificultades cuando se intentaba realizar un recorte cuya región de interés excedía las dimensiones del frame original. Esto generaba accesos inválidos a memoria en la GPU. La solución implementada consistió en añadir validaciones de límites tanto antes del lanzamiento del kernel como dentro del mismo, verificando que las coordenadas calculadas no superaran el ancho ni el alto de la imagen original. Con esta corrección, el recorte funciona de manera segura y estable para diferentes tamaños de región.

8. Tabla de Evaluación Interna

Criterio	Dylan	Jonathan	Steeven
Desarrollo de proyectos con paralelización	10 %	10 %	10 %
Documentación del aporte individual	10 %	10 %	10 %
Análisis de rendimiento	10 %	10 %	10 %
Código limpio y funcional	10 %	10 %	10 %
Trabajo en equipo	10 %	10 %	10 %

9. Repositorio de Ejecución en Google Colab

Con el fin de facilitar la verificación del correcto funcionamiento de los algoritmos implementados, se dispuso el desarrollo completo del taller en un entorno de ejecución en línea mediante Google Colab.
En este repositorio se encuentran incluidos:

- Implementación de los cinco filtros solicitados utilizando CUDA.
- Configuración de la malla de ejecución (*grid* y *blocks*).
- Gestión de memoria entre host y device.
- Manejo de límites para evitar accesos inválidos a memoria.
- Lectura del video de entrada y extracción secuencial de frames.

- Procesamiento frame a frame mediante GPU.
- Reconstrucción del video final procesado.
- Visualización comparativa entre frames originales y frames procesados.

El notebook permite ejecutar cada uno de los ejercicios de manera independiente, empleando un video de prueba para evidenciar el comportamiento de los algoritmos ante diferentes características visuales.

`https://colab.research.google.com/drive/1bkDK0jKYWijOi_kPIlbCQVd7Hnb24Hgr?usp=sharing`

10. Conclusiones Generales

A lo largo del desarrollo del presente taller fue posible evidenciar el impacto positivo del uso de técnicas de paralelización mediante GPU en el procesamiento de información visual. La implementación de los diferentes filtros utilizando CUDA permitió distribuir el cálculo de cada píxel entre múltiples hilos de ejecución, aprovechando el paralelismo masivo inherente a este tipo de arquitecturas.

Se observó que los filtros de tipo puntual, como la inversión de colores y la binarización, presentan un alto grado de paralelización debido a la independencia entre píxeles, lo que se traduce en tiempos de ejecución considerablemente menores en comparación con su implementación secuencial en CPU. Por otro lado, filtros basados en convolución como el Blur y el operador Sobel, aunque requieren mayor carga computacional por píxel, también se benefician significativamente de la ejecución simultánea en GPU.

Adicionalmente, se identificó que el manejo adecuado de los límites de los frames constituye un aspecto fundamental para garantizar la estabilidad de los kernels y evitar accesos inválidos a memoria durante la ejecución en el dispositivo. La inclusión de validaciones condicionales permitió asegurar el correcto funcionamiento de los algoritmos sin afectar de manera significativa el rendimiento general.

El presente laboratorio permitió extender los algoritmos desarrollados para imágenes individuales al procesamiento de video, demostrando que los mismos kernels CUDA pueden aplicarse de manera eficiente sobre secuencias completas de frames. Este enfoque evidencia cómo los métodos de procesamiento de imágenes pueden escalar naturalmente hacia aplicaciones de procesamiento de video mediante técnicas de paralelización en GPU.

En conclusión, el uso de CUDA representa una herramienta eficiente para la optimización de algoritmos de procesamiento de imágenes y video, permitiendo reducir tiempos de ejecución y mejorar la escalabilidad de los métodos implementados frente a volúmenes más grandes de información visual.